Gram Setu — Technical Design Document

Oct 15, 2026 · @sampurn gaur

1. Overview

Gram Setu (ग्राम सेतु) is a working prototype of a rural grievance-redressal platform: citizens log in by phone OTP, file complaints, and track them, while admins triage them with help from a three-stage Gemini-based AI pipeline. This document describes the code in the supplied ZIP (`GramSetu_Clean`, about 5,600 lines), not the full planned product.

What the prototype does today

- Passwordless citizen login with a 6-digit OTP and an HttpOnly session cookie.
- Complaint intake (text, GPS, photo as a data URI) with a generated ID of the form `GRV-YYYY-00NNNN`.
- Stage 1 AI: extracts category, problem type, affected resource, scope, duration and severity. Routing, department, officer, priority and SLA hours then come from fixed per-category rules.
- Stage 2 AI: groups differently worded complaints into one underlying "problem cluster".
- Stage 2.5 AI: validates a cluster against photos, location, time and resource evidence and stores a separate validated confidence.
- Admin console: stats, filters, complaint editing, citizen directory, cluster views, audit log and CSV export.

Design principles visible in the code

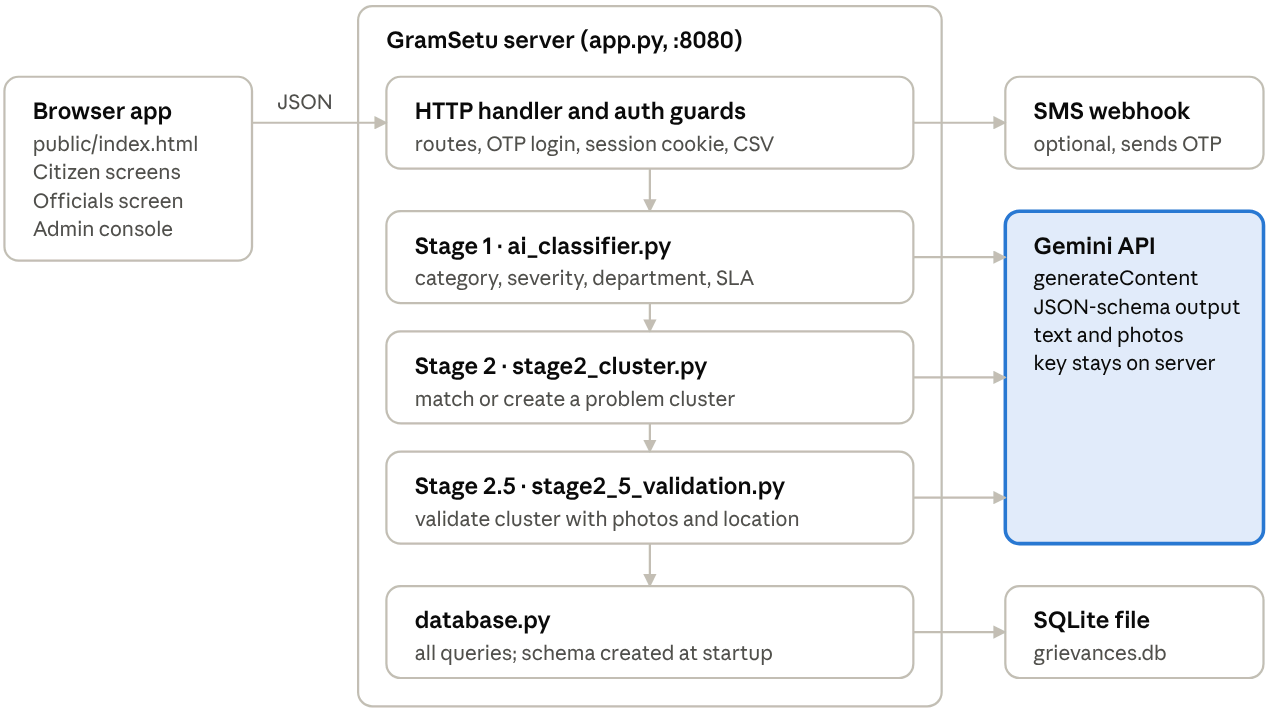
- AI is advisory. The original complaint text is never overwritten, and AI output is stored in separate tables.
- Every AI stage has a rule-based fallback, so complaint registration never depends on Gemini being up.
- The Gemini API key stays server-side. Only complaint text and village/district go to Gemini; name and phone are not sent.
- Stages are conservative: ambiguous matches create a new cluster rather than force a merge (auto-match needs at least 0.75 confidence).

Non-goals of this build: Stage 3 (dynamic priority recommendation), a real notification channel, department-officer accounts, multilingual UI, and production hosting. Several of these appear in the original plan but have no code yet (section 7).

2. System architecture

The prototype is a single-process monolith: a Python standard-library HTTP server serves one static HTML page and a JSON API, stores everything in a local SQLite file, and calls the Gemini REST API synchronously. It has no third-party Python dependencies. This differs from the stack in the original plan (React, Tailwind, Flask, PostgreSQL), which is not in this ZIP.

One Python process serves the UI, runs three AI stages and writes to SQLite



system architecture · 1 process, 3 external services

The browser talks only to the server. The server is the sole caller of Gemini, the SMS webhook and SQLite, and the Gemini key never reaches the browser.

Layer	Choice in the code	Notes
Frontend	One `public/index.html` (2,746 lines), vanilla JS, Tailwind from a CDN script, Google Fonts (Baloo 2, Mukta)	Five screens: home, report, track, officials, admin
Web/API server	`http.server` + `socketserver.TCPServer` on port 8080 (`app.py`)	Single-threaded; serves static files and `/api/*`
Persistence	SQLite file `grievances.db` (`database.py`)	Schema created at startup; seeded with sample complaints when empty
AI	Gemini `generateContent` REST, model `gemini-3.5-flash-lite` by default	Structured JSON output; rule-based fallback per stage
Auth	Phone OTP, HMAC-hashed OTP, SHA-256-hashed session token in cookie `gramsetu_session`	OTP delivery via an optional SMS webhook
Config	`.env` file or environment variables	`GEMINI_API_KEY`, `GEMINI_MODEL`, `GRAMSETU_ENV`, `GRAMSETU_OTP_SECRET`, `GRAMSETU_SMS_WEBHOOK_URL`, `GRAMSETU_ADMIN_PHONE`

Repository layout

File	Responsibility
`app.py`	HTTP handler, routing, auth guards, CSV export, SMS webhook call
`database.py`	SQLite schema, auth tables, grievance, cluster, validation and admin queries
`ai_classifier.py`	Stage 1: Gemini analysis, rule-based fallback, category-to-department/SLA map, ID generator
`stage2_cluster.py`	Stage 2: candidate ranking, Gemini MATCH/NEW decision, cluster create/join

File	Responsibility
`stage2_5_validation.py`	Stage 2.5: multimodal evidence validation and fallback
`STAGE*_AI.md`, `README.md`	Design notes per AI stage and run instructions
`run_mac_linux.sh`, `run_windows.bat`	Launchers

3. Components

Frontend (`public/index.html`). A single-page app that swaps `<section id="screen-*">` blocks. Citizens use Home, Report (text, GPS capture, photo upload, and a voice button that is only a simulation: it types three canned Hindi sentences and calls no speech API) and Track. The Officials screen and the Admin console are in the same file. The Officials screen and the citizen confirm/dispute buttons only change in-browser state and never call the server (section 7). Auth state comes from `GET /api/auth/me`, and the session lives only in the HttpOnly cookie. Complaint lookups always go to the server; `localStorage` is cleared on logout.

API server (`app.py`). One `GrievanceAPIHandler` class with `do_GET` and `do_POST`. Guards are `require_user()` and `require_admin()`, which resolve the cookie to a user and check `role`. It also builds the CSV export and posts OTPs to the SMS webhook.

Data layer (`database.py`). Opens a new SQLite connection per call. It holds OTP rate limiting (60 s resend gap, 5 per rolling hour, 5 verify attempts, 5-minute expiry), 7-day sessions, and all CRUD for grievances, AI analyses, clusters, validations and the admin audit log.

AI pipeline. Three sequential stages run inside the complaint-submit request. Each stage is a pure function over the database plus one Gemini call, and each falls back to local rules.

Stage	Module	Input	Output	Fallback
1. Understand	`ai_classifier.py`	Complaint text, village, district, optional category hint	Category, problem type, affected resource, scope, duration, affected people, severity, confidence	Keyword match over seven categories, English and Hindi
2. Cluster	`stage2_cluster.py`	Stage 1 context and up to 12 candidate clusters, ranked locally to the top 6	MATCH or NEW, cluster ID, confidence, reason	Local similarity (token overlap, sequence ratio, structured fields); auto-match at 0.78 or more
2.5. Validate	`stage2_5_validation.py`	Cluster plus up to 8 member reports, with photos if they are data URIs	CONFIRMED, POSSIBLE or CONFLICT, five evidence scores, asset match, conflicts	Same-village and same-category heuristic

Stage 2.5 only runs once a cluster has at least two members. Neither Stage 2 nor 2.5 changes the complaint's priority.

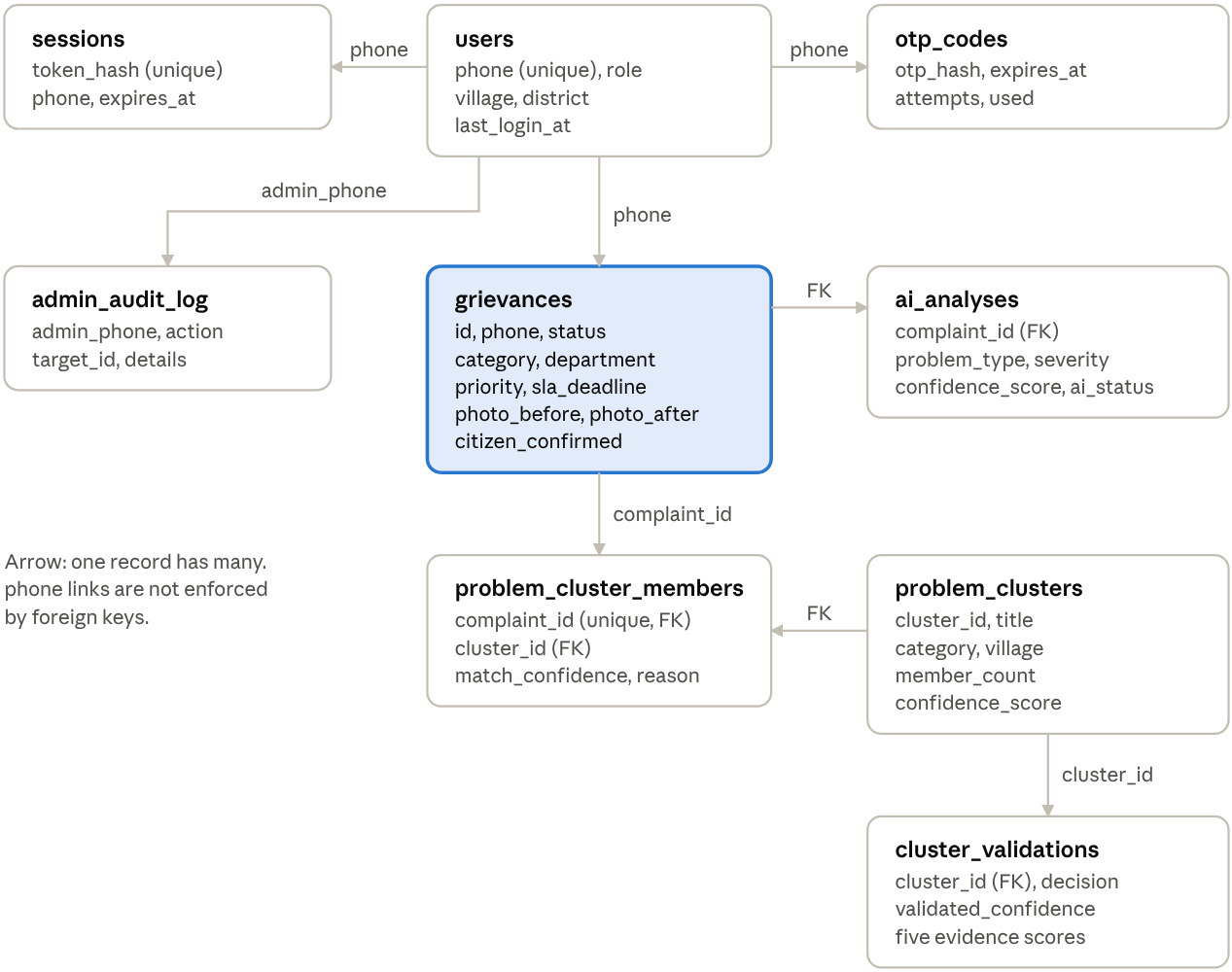
Routing and SLA rules (fixed table in `ai_classifier.py`). Stage 1 chooses the category; everything below is then a lookup, not AI.

Category	Department	SLA (hours)	Default priority
Electricity	State Electricity Distribution Company (DISCOM)	24	High
Healthcare	CMO and District Health Society	24	High
Water	Rural Water Supply and Sanitation (Jal Shakti / PHED)	48	High
Agriculture	Minor Irrigation and Agriculture Dept	48	High
Sanitation	Gram Panchayat / Swachh Bharat Mission Cell	72	Medium
Government scheme	District Collectorate and Social Welfare	120	Medium
Roads	PWD / Gram Panchayat	168	Medium
Other, and categories with no rule (Education, Street Lighting and others)	Gram Panchayat / Block Development Office	120	Medium, or Standard if the fallback matched nothing

4. Data model and API

The database has nine tables in one SQLite file. Complaints and AI output are separate: `grievances` holds what the citizen and officials wrote, and every AI result lives in its own table. A complaint belongs to at most one cluster (`complaint_id` is unique in `problem_cluster_members`), and a cluster accumulates validation runs over time.

Complaints, AI output and clusters are stored in separate tables



entity relationships · 9 tables

Table	Purpose	Key columns
`users`	Citizen and admin accounts	`phone` (unique), `role`, `village`, `district`
`otp_codes`	Hashed one-time codes	`phone`, `otp_hash`, `expires_at`, `attempts`, `used`
`sessions`	Hashed session tokens	`token_hash` (unique), `phone`, `expires_at`

Table	Purpose	Key columns
`grievances`	The complaint and its lifecycle	`id` (`GRV-...`), `phone`, `category`, `department`, `priority`, `sla_deadline`, `status`, `photo_before`, `photo_after`, `citizen_confirmed`
`ai_analyses`	Stage 1 output per complaint	`complaint_id`, `problem_type`, `affected_resource`, `scope`, `severity`, `confidence_score`, `ai_provider`, `ai_status`, `raw_analysis`
`problem_clusters`	One row per underlying problem	`cluster_id`, `title`, `category`, `member_count`, `confidence_score`, `last_reason`
`problem_cluster_members`	Complaint-to-cluster link	`cluster_id`, `complaint_id` (unique), `match_confidence`, `match_reason`
`cluster_validations`	Stage 2.5 history	`cluster_id`, `stage2_confidence`, `validated_confidence`, `decision`, five evidence scores, `asset_match`, `review_required`
`admin_audit_log`	Admin actions	`admin_phone`, `action`, `target_id`, `details`

`phone` is the join key between users, sessions, OTPs and grievances. It is a plain text column with no foreign key. Timestamps are stored as ISO text, and photos are stored inline as data-URI text in `grievances`.

Complaint status values: `New`, `In Progress`, `Resolved`, `Closed`. A citizen who disputes a resolution sets the status back to `In Progress` and priority to `Emergency Escalation`. Priority values are `Low`, `Medium`, `High`, `Emergency Escalation` (the fallback can also write `Standard`).

HTTP API

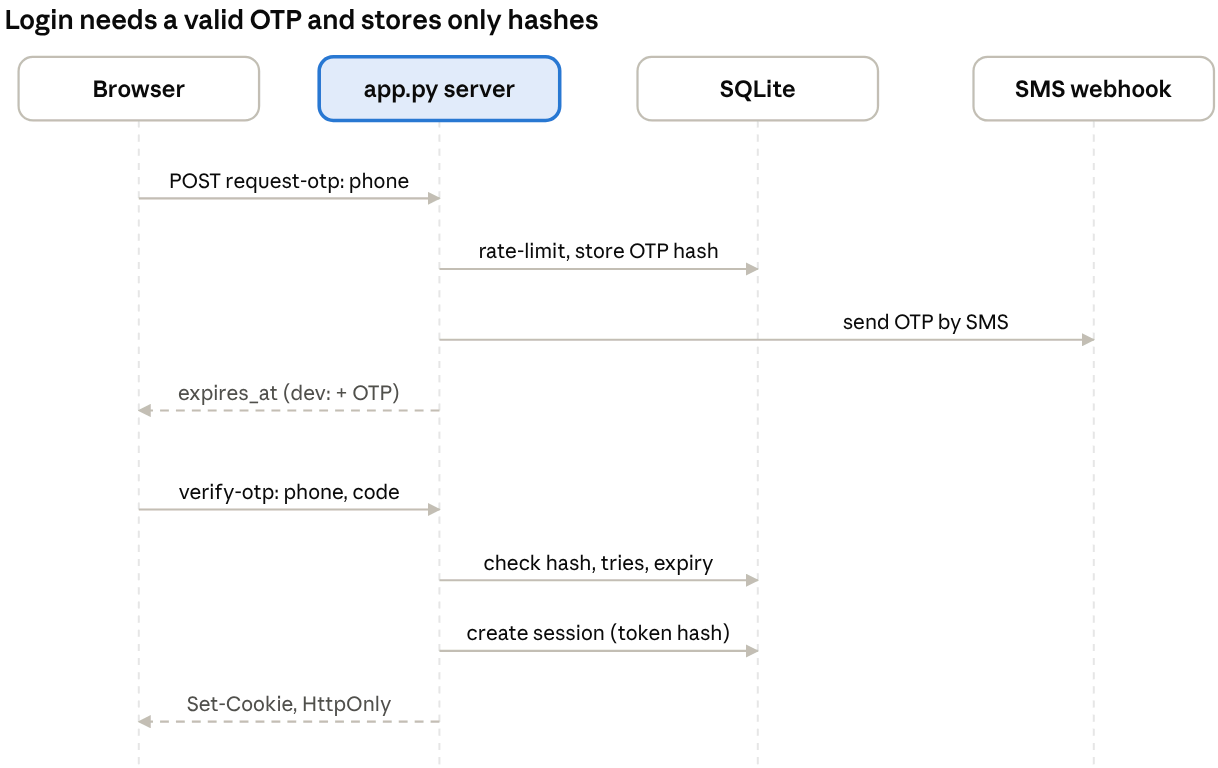
Method and path	Access	Purpose
`POST /api/auth/request-otp`	Public	Create and send an OTP; upsert the user
`POST /api/auth/verify-otp`	Public	Verify OTP; set session cookie
`POST /api/auth/logout`, `GET /api/auth/me`	Cookie	End session; read current user
`POST /api/ai/classify`	Citizen	Live Stage 1 preview for a text
`POST /api/grievances`	Citizen	File a complaint; runs Stages 1, 2, 2.5
`GET /api/grievances`, `GET /api/grievances/{id}`	Citizen (own) or admin	List or fetch complaints; not-owner returns 404
`POST /api/grievances/{id}/resolve`	**None**	Official resolution update (before/after photo, notes)
`POST /api/grievances/{id}/confirm`	**None**	Citizen confirmation or dispute
`GET /api/admin/stats`, `filters`, `complaints`, `users`, `audit`, `database`, `ai-status`	Admin	Dashboard data
`GET /api/admin/export/complaints`	Admin	CSV export (audited)
`GET /api/admin/clusters`, `/clusters/{id}`, `/complaints/{id}/ai`	Admin	Cluster list and detail; Stage 1 analysis
`POST /api/admin/complaints/{id}`	Admin	Update status, priority, officer, notes (audited)
`POST /api/admin/stage2/cluster`, `/stage2_5/validate`, `/stage2_5/history`	Admin	Re-run clustering or validation; read history

Responses use `{status, data|message}` JSON. Errors carry a plain message and an HTTP status (400, 401, 403, 404, 429, 503).

5. Key flows

Three flows carry the system: login, complaint submission, and the complaint lifecycle. Each diagram follows the code paths in `app.py` and `database.py`.

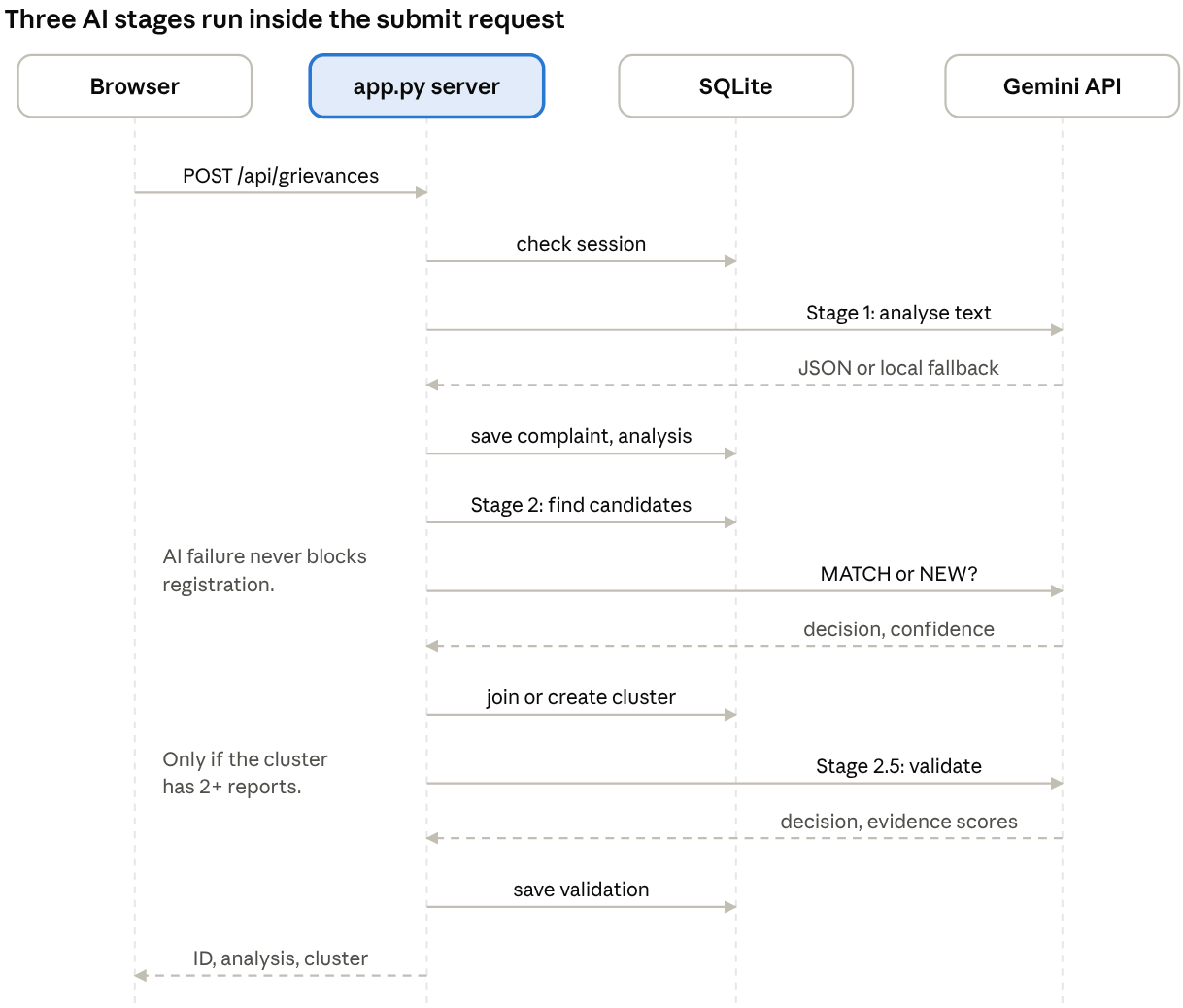
Phone OTP login



phone OTP login · sequence, 4 participants

A phone number is enough to create an account; the OTP proves ownership. In development mode, which is the default, the OTP is returned in the response. In production it goes out by the SMS webhook, and a failed send returns 503.

Complaint submission

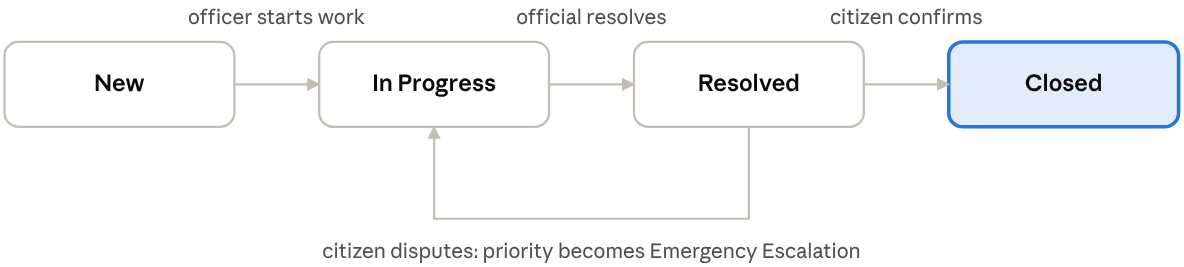


complaint submission · sequence, 3 AI stages

The server saves the complaint before Stage 2 runs, and Stage 2 and 2.5 sit inside a try block, so they cannot fail a registration. Stage 1 returns the rule-based result on any Gemini error. The cost is latency: the citizen waits for every AI call, and the single-threaded server blocks other users meanwhile (finding 5).

Complaint lifecycle

Closure follows citizen confirmation, but admin edits can bypass it



Admin edits (POST /api/admin/complaints/:id) can set any of the four statuses directly.
The resolve and confirm endpoints accept calls from anyone today (finding 1, section 7).

complaint status lifecycle · 4 states, 1 loop

A citizen who confirms sets the status to Closed. A citizen who disputes sets it back to In Progress and raises priority to Emergency Escalation. Those transitions exist in `database.py`, but the browser never calls them and no timer escalates an overdue SLA.

6. Security, configuration and operations

Controls that are implemented

- OTPs are stored as HMAC-SHA256 hashes, expire in 5 minutes, allow 5 wrong attempts, and are rate-limited to one per 60 seconds and 5 per hour per phone.
- Session tokens are random 32-byte values; only their SHA-256 hash is stored. The cookie is `HttpOnly; SameSite=Lax` with a 7-day life, and logout deletes the server-side session.
- Admin role is granted only by the `GRAMSETU_ADMIN_PHONE` environment variable at startup. There is no promotion endpoint.
- Every `/api/admin/*` route calls `require_admin()`. Citizens can read only complaints whose `phone` matches theirs, and a foreign ID returns the same 404 as an unknown one.
- Admin edits and CSV exports are written to `admin_audit_log`. The AI-status endpoint reports only whether a key is loaded, never the key.
- Gemini receives complaint text and village/district only. Name and phone are not sent.

Configuration

Variable	Default	Effect
`GRAMSETU_ENV`	`development`	Anything other than `production` returns the OTP in the API response (`dev_otp`)
`GRAMSETU_OTP_SECRET`	`dev-only-change-this-secret`	HMAC key for OTP hashes
`GRAMSETU_SMS_WEBHOOK_URL`	empty	POST target for `{phone, message}`; required in production or OTP requests return 503
`GRAMSETU_ADMIN_PHONE`	empty	Comma-separated 10-digit admin phones
`GEMINI_API_KEY`	empty	Enables Gemini for all three stages; empty means rule-based only
`GEMINI_MODEL`	`gemini-3.5-flash-lite`	Model name; the old `gemini-2.5-flash-lite` value is rewritten to this

Running it. `python app.py` starts the server on port 8080 (hard-coded) and creates and seeds `grievances.db` next to the code. `.env` is loaded by `app.py` and again by `stage2_5_validation.py`, and the ZIP excludes the database and any `.env`. There is no Dockerfile, TLS, process manager, backup job, health endpoint, structured logging or test suite. Errors go to `print`.

7. Gaps found in the code and recommendations

The two most serious problems are that anyone can resolve or reopen any complaint without logging in, and that the officials workflow never reaches the server. Findings are from reading the code; nothing here was run under load or attack.

#	Finding	Where	Severity	Fix
1	`POST /api/grievances/{id}/resolve` and `/confirm` have no auth, no ownership check and no audit entry. Anyone who guesses an ID can close, reopen or escalate a complaint and set its officer and photos.	`app.py` do_POST	High	Require login; `/confirm` only for the owning citizen; `/resolve` only for an assigned official or admin; audit both
2	The Officials screen and the citizen "confirm / dispute" buttons change only the in-browser `grievances` array and never call the API. The core promise (no closure without citizen confirmation, auto-escalation on dispute) is therefore not enforced anywhere.	`index.html` `handleOfficialSubmit`, `confirmResolutionYes/No`	High	Wire both to the server and enforce the state machine there (see below)
3	Development mode is the default and returns the OTP in the response. The default OTP secret is a public string.	`app.py`, `database.py`	High	Default to production behaviour; fail startup if the secret is unset or default
4	Complaint IDs are `GRV-YYYY-00` plus a random 4-digit number: 9,000 values, guessable (see finding 1) and collision-prone. A collision breaks the primary key and the submit request fails.	`generate_complaint_id`	Medium	Use a sequence or a random 8+ character token; retry on conflict
5	The server is single-threaded and the Gemini calls are synchronous. One submit can wait through Stage 1 (up to 60 s, with retries), Stage 2 (60 s) and Stage 2.5 (90 s) while every other request is blocked.	`app.py` `TCPServer`, stage modules	Medium	Use `ThreadingHTTPServer` at minimum; better, save the complaint, return the ID, and run AI in a background worker
6	Static-file branch joins the raw URL path to `PUBLIC_DIR`, so `/../x.html`, `.js` or `.css` files outside `public/` can be read by a client that does not normalize paths.	`app.py` do_GET	Medium	Resolve and check the path stays under `PUBLIC_DIR`, or drop the custom branch
7	SLA deadlines are stored but nothing watches them. There is no scheduler, no timeout escalation and no notification (SMS, email or web) other than the OTP.	whole codebase	Medium	Add a scheduled job for SLA breaches and a notification service
8	Priority and department come from a fixed per-category table, so Stage 1 AI category effectively drives routing and SLA even though the docs call AI advisory. Stage 3 (dynamic priority, official override with history) is not built.	`ai_classifier.py`	Medium	Store AI priority and official priority separately; let officials override with an audit trail

#	Finding	Where	Severity	Fix
9	No request-size limit and no JSON error handling: a malformed body raises inside the handler. Photos are base64 text inside SQLite, and the complaint list loads every row, photos included, then filters by phone in Python.	`app.py`, `database.py`	Medium	Cap body size; return 400 on bad JSON; store photos as files or object storage; filter in SQL
10	Session cookie lacks `Secure`. `Access-Control-Allow-Origin: *` is set on JSON responses, and there are no CSRF tokens (SameSite=Lax is the only protection). OTP comparison is not constant-time.	`app.py`, `database.py`	Low	Add `Secure`; restrict CORS; use `hmac.compare_digest`
11	Complaint text is placed in Gemini prompts, and photos are sent to Gemini in Stage 2.5. Output is schema-constrained, but photos may show people or other personal data.	`stage2_*`	Low	Add a consent notice; minimise or blur images; log what is sent
12	Dead or duplicate code: unreachable GET-style cluster routes inside `do_POST`, a `.env` loader in two files, a client-side classifier and sample data duplicating server logic, and a simulated voice button. Tailwind is loaded from a third-party script URL, which hurts offline and low-bandwidth use.	`app.py`, `index.html`	Low	Remove; bundle CSS locally; add real speech-to-text

Complaint lifecycle to enforce. The plan describes Submitted, Received, Assigned, In Progress, Resolved, then Citizen Confirmation. The code has four statuses. A server-side state machine should own every transition (diagram in section 5) and reject out-of-order calls.

Suggested path to the planned product

1. Fix findings 1 to 4 first; they are small and remove the worst exposure.
2. Move to a framework and database that match the plan (Flask or FastAPI with PostgreSQL), keeping the stage modules unchanged behind a service interface.
3. Add a background worker for AI stages and SLA timers, and an SMS/email/web notification service.
4. Build official accounts, Stage 3 priority with override history, and a real speech-to-text and translation step.
5. Add tests for the OTP flow, access control per endpoint, the status state machine and the three AI fallbacks.